Technical Status Report: Antigravity DB

Date: July 31, 2026 | Audit Type: Deep Technical & Security Code Review

1. ARCHITECTURE OVERVIEW

- Backend Structure (FastAPI):

The core backend is a monolithic entry file at backend/main.py (~1,871 lines). Core modules include backend/model_router.py (Circuit breaker), backend/auth.py (JWT/OIDC), backend/sql_validator.py (AST tenant filtering), backend/hybrid_retriever.py (BM25 + ChromaDB), and backend/schema_graph.py (FK graph).

- Frontend Structure:

A dual-frontend setup exists: a legacy Streamlit UI at frontend/app.py (31KB) and a modern React SPA at web/src (Vite + Zustand chatStore.js + Vanilla CSS index.css).

- Data Flow Pipeline:

User Input -> FastAPI -> auth.py -> query_intelligence.py -> hybrid_retriever.py -> schema_graph.py -> model_router.py -> SQL Generation -> sql_validator.py -> PostgreSQL Pool -> Response / SSE Stream.

2. MODEL ROUTING & LLM LAYER

- Multi-Tier Routing Chain:

Primary: Groq API (llama-3.3-70b-versatile). Fallback 1: Google Gemini (gemini-2.0-flash). Fallback 2: Ollama/Qwen (qwen2.5-coder:7b). BYO API keys configured in settings override this chain.

- Circuit Breaker Behavior:

Hard failures (HTTP errors/timeouts) increment failure counters. 3 failures trip circuit to 'open'. Primary model attempts recovery after 300 seconds.

- Critical Breaker Risk:

Non-primary fallbacks (Gemini/Qwen) do NOT have auto-recovery implemented (should_attempt_recovery returns False). Once open, fallbacks remain permanently disabled until restart.

- Context Window Safeguards:

OLLAMA_NUM_CTX is set to 8192. Prompt lengths are not dynamically checked with token counters (e.g. tiktoken), risking silent API payload truncation on large schema dumps.

3. DATABASE & MULTI-TENANCY

- Multi-Tenant Data Security:

Tenant isolation is enforced purely at the application logic layer inside backend/sql_validator.py (validate_org_security). No Database-level Row-Level Security (RLS) is applied on PostgreSQL tables.

- ChromaDB Embedding Status:

ChromaDB imports gracefully fall back if native C++ build tools are missing. However, there is no automated background sync worker to re-index embeddings when Postgres DDL schema changes occur.

- SQL Injection Surface:

LLM-generated queries are sanitized via AST parsing. Dynamic string building exists in schema_graph.py and hybrid_retriever.py for internal metadata queries.

4. STREAMING & REALTIME

- SSE Implementation:

Realtime streaming uses FastAPI StreamingResponse and SSE. Generator loops yield tokens dynamically.

- Weaknesses & Memory Leaks:

Abandoned client connections can leave generator tasks hanging until httpx network timeouts expire (up to 300s). Lack of periodic SSE ping/heartbeat events risks termination by proxies/ALBs.

5. SECURITY

- Authentication Vulnerability:
REQUIRE_AUTH in backend/auth.py defaults to 'false'. Unless explicitly configured to 'true' in production, all routes operate completely unauthenticated.

- Secrets & Credentials:
Plaintext credentials exist in .env and .env.save files. API key logging must be audited.

- Rate Limiting:
Rate limiting relies on slowapi (get_remote_address) per IP. Lacks tenant-level rate limiting.

6. FRONTEND STATE

- State & Styling:
React frontend uses Zustand (chatStore.js) and 70KB index.css. Visual components are modularized.

- Architecture Risk:
Coexistence of Streamlit (app.py) and React (web/src) creates drift and code maintenance friction.

7. DEPLOYMENT & INFRASTRUCTURE

- Containers & Cloud:
Dockerfiles present for backend and web. docker-compose.yml available for local composition. ECS task definitions (deploy/ecs-task-definition.json) and Terraform modules (deploy/terraform) exist.

- Pipeline:
GitHub Actions deploy.yml workflow present. Requires secret provisioning before live runs.

8. TECHNICAL DEBT & KNOWN RISKS

- Monolithic backend/main.py:
1,871 lines combining routes, database connections, schema caching, and prompt pipelines.

- No Row Level Security:
Tenant isolation entirely dependent on app-layer SQL regex/AST filtering.

- Silent Feature Degradation:
Embedding retrieval silently disabled on missing build dependencies.

9. TEST COVERAGE

- Coverage Status:
0% Automated Test Coverage. Zero unit or integration tests exist in the codebase.

- Quality Assurance:
Entire system relies on manual interactive testing.